Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 1:

The Operating System as an Intermediate Layer between the User and the Hardware

Course: Operating Systems

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. *Relitman Patricia*

Chişinău – 2026

Contents

1 OBJECTIVE **2**

2 INITIAL SETUP **2**

3 PART 1 – FILES AND DIRECTORIES **2**
- 3.1 Exploring the File System . . . 2
- 3.2 Creating, Copying, Renaming and Deleting Files . . . 2
- 3.3 File Permissions . . . 3

4 PART 2 – PROCESSES **3**
- 4.1 Listing the Processes . . . 3
- 4.2 Background Processes . . . 4
- 4.3 The `/proc` Pseudo-File System . . . 4

5 PART 3 – MEMORY **4**

6 PART 4 – DEVICES AND STORAGE **5**
- 6.1 Disk Usage . . . 5
- 6.2 Devices as Files . . . 5

7 CONCLUSION **6**

1 OBJECTIVE

This laboratory work gives an overview of the operating system (OS) as the intermediate layer between the user and the hardware. Four of the main responsibilities of the OS are explored through shell commands on a Linux machine: the management of **files**, of **processes**, of **memory** and of **devices**. The commands are executed in order to observe how each of these mechanisms works and which actions it implies, and the output of every command is interpreted in the corresponding section.

2 INITIAL SETUP

As a starting point, a working directory is created for all the exercises and three commands identify the system on which the work is done.

```
mkdir -p ~/Documents/cursuri/sisteme-operare/os-lab1 && cd ~/
    Documents/cursuri/sisteme-operare/os-lab1
whoami       # who am I to the OS
uname -a     # which kernel is managing this machine
uptime       # how long has this OS been running
```

Listing 1: Creation of the working environment and system identification.

`mkdir -p` creates the whole chain of directories (and does not complain if some already exist), after which `cd` enters the new directory. `whoami` prints the current user, `uname -a` describes the kernel that manages the machine, and `uptime` shows for how long the system has been running.

3 PART 1 – FILES AND DIRECTORIES

3.1 Exploring the File System

```
pwd
ls -la /
ls -la ~
cd /etc && ls | head
```

Listing 2: Navigation through the file system hierarchy.

The first command prints the working directory, which here is `/home/lungus/Documents/cursuri/sist`. Then `ls -la /` lists the content of the root directory, including hidden entries, together with the access rights, the owner, the size in bytes and the other attributes of each entry. `ls -la ~` shows the same information for the home directory. Finally, `cd /etc && ls | head` moves to `/etc` and uses a *pipeline*: the output of `ls` is passed to `head`, which keeps only the first 10 lines.

3.2 Creating, Copying, Renaming and Deleting Files

```
cd ~/Documents/cursuri/sisteme-operare/os-lab1
mkdir demo && cd demo
echo "hello operating system" > note.txt
cp note.txt copy.txt
mv note.txt renamed.txt
```

```
ls -l
rm renamed.txt
```

Listing 3: Basic file operations in the `demo` directory.

The shell returns to the laboratory directory, creates the directory `demo` and enters it. The `echo` command prints a string, and the redirection operator `>` writes it to the file `note.txt`. The file is then copied as `copy.txt` with `cp` and renamed to `renamed.txt` with `mv`. Finally, `ls -l` shows the content of `demo` (`renamed.txt` and `copy.txt`) and `rm` deletes `renamed.txt`, leaving only `copy.txt`.

3.3 File Permissions

```
ls -l copy.txt
chmod 600 copy.txt
ls -l copy.txt
chmod 644 copy.txt
```

Listing 4: Changing the access rights of `copy.txt`.

The permission string of `copy.txt` changed in the following way:

```
-rw-rw-r--
-rw-------
-rw-r--r--
```

Listing 5: Permission strings of `copy.txt` after each step.

Table 1: Permissions of `copy.txt` before and after each `chmod`.

Step	Owner	Group	Others
Initial (`-rw-rw-r-`)	`rw-`	`rw-`	`r-`
`chmod 600` (`-rw----`)	`rw-` (6)	`--` (0)	`--` (0)
`chmod 644` (`-rw-r-r-`)	`rw-` (6)	`r-` (4)	`r-` (4)

In every digit of the octal notation, read = 4, write = 2 and execute = 1. For `chmod 600` the owner receives $6 = 4 + 2$, i.e. read and write, while the group and the others receive 0 and therefore have no access to the file. For `chmod 644` the owner keeps read and write, and the group and the others may only read (4). During these changes the owner and group columns of the listing remained the same: `chmod` modifies only the access rights, not the ownership of the file.

4 PART 2 – PROCESSES

4.1 Listing the Processes

```
ps aux | head
ps aux | wc -l
top
```

Listing 6: Observing the running processes.

`ps aux | head` displays the first 10 lines of the process list. `ps aux | wc -l` counts the lines of that list; on the test machine it reports 298 lines, which is the number of running processes plus the header line. `top` presents a continuously updated list of processes with their PID, user, memory footprint, processor usage, running time and other data.

4.2 Background Processes

```
sleep 300 &
jobs
ps -ef | grep sleep
kill <PID>
```

Listing 7: Starting, finding and terminating a background process.

The command `sleep 300` suspends for 300 seconds, and the character `&` sends it to the background, so the shell returns immediately and prints the PID of the new process, which is 5752 in this session. `jobs` lists the background jobs of the shell with their status. `ps -ef` lists all the processes and `grep sleep` keeps only the lines containing the word `sleep`, which shows the process with its data. Finally, `kill 5752` terminates it.

4.3 The /proc Pseudo-File System

```
sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
```

Listing 8: Inspecting a process through `/proc`.

As in the previous example, a process sleeping for 300 seconds is started in the background. The special variable `$!` holds the PID of the last background process and is saved in the variable `PID`; in this run it is 5842. The directory `/proc/$PID/` is a pseudo-file system through which the kernel exposes information about the process: `ls` shows its entries, and `cat /proc/$PID/status | head -20` prints the first 20 lines of the `status` file. There the process appears in the state `S (sleeping)`, as expected for a process that is waiting for its timer. The process is then terminated with `kill $PID`.

5 PART 3 – MEMORY

```
free -h
cat /proc/meminfo | head -6
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

Listing 9: Observing the memory of the system and of a process.

`free -h` shows the use of the physical memory and of the *swap* space in human-readable units. On the host machine the output is:

```
               total        used        free      shared  buff/cache   available
Mem:            15Gi       4.3Gi       8.4Gi       754Mi       3.8Gi        11Gi
Swap:          2.0Gi          0B       2.0Gi
```

Listing 10: Output of `free -h`.

Of 15 GiB of RAM, 4.3 GiB are used by programs, 3.8 GiB are occupied by buffers and cache, and 11 GiB are available to new programs. The swap space of 2.0 GiB is not used at all (`0B`), which shows that the physical memory is sufficient for the current load. The command `cat /proc/meminfo | head -6` prints the first lines of the kernel's detailed memory statistics, which contain the same kind of information.

For the sleeping process, `grep VmRSS /proc/$PID/status` reports `2132kB`. `VmRSS` (resident set size) is the part of the process' memory that is currently held in physical memory (RAM); the very small value is natural for a process that only waits.

6 PART 4 – DEVICES AND STORAGE

```
df -h
lsblk
du -sh ~/Documents/cursuri/sisteme-operare/os-lab1
ls -l /dev | head
mount | head
```

Listing 11: Observing the storage devices and the mounted file systems.

6.1 Disk Usage

`df -h` shows the mounted file systems with their size and usage:

```
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           1.6G  1.9M  1.6G   1% /run
efivarfs        374K  274K   96K  75% /sys/firmware/efi/efivars
/dev/sda2       109G   74G   30G  72% /
tmpfs           7.7G   28M  7.7G   1% /dev/shm
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
/dev/sda1       511M   20M  492M   4% /boot/efi
tmpfs           1.6G  244K  1.6G   1% /run/user/1000
```

Listing 12: Output of `df -h`.

The root directory `/` is mounted from `/dev/sda2`, a partition of the drive on which the system is installed; it has 109 G, of which 72% is used. Another partition of the same drive, `/dev/sda1` (511 M), is mounted on `/boot/efi` and is allocated to the EFI boot structure. The `tmpfs` entries are file systems kept in memory (for example `/run` and `/dev/shm`), so they do not occupy disk space. The command `du -sh` reports that the laboratory directory occupies only `12K`.

6.2 Devices as Files

`ls -l /dev | head` lists the devices, which the system presents as files. One of them is the console:

```
crw--w----   1 root tty         5,     1 Oct  4 09:38 console
```

Listing 13: A device file from `/dev`.

The first character, `c`, shows that this is a *character device*, and the pair of numbers `5, 1` are its major and minor numbers, which identify the driver in the kernel. The entry illustrates a key philosophy of UNIX-like systems: "everything is a file". Storage, memory, devices and peripherals are represented as files through which the OS communicates with them, so the same operations (open, read, write) can be used for all of them.

Finally, `mount | head` lists the file systems attached to the directory tree, among them `/dev/sda2`, the storage used for the user's data and programs.

7 CONCLUSION

In this laboratory work the operating system was explored as the layer that hides the hardware from the user and manages its resources. Through simple shell commands the four main areas of resource management were observed in practice.

In the file system, the directory hierarchy was explored and files were created, copied, renamed and deleted; the permission experiment showed how `chmod` changes the read, write and execute rights of the owner, the group and the others using the octal notation (600 and 644), without changing the ownership. In process management, background processes were started with `&`, found with `ps` and `grep`, inspected through `/proc` and terminated with `kill`. The memory commands showed the distribution of RAM and swap on the machine and the small resident memory (`VmRSS`, 2132 kB) of an idle process. Finally, the storage commands showed how the disk is divided into partitions mounted in the directory tree, and the `/dev` directory showed that devices are accessed as files.

The common feature of all four areas is that the OS gives the user a uniform interface – files, processes and pseudo-files such as `/proc` and `/dev` – through which the state of the system can be read and controlled with the same set of tools.